

Low-Level Design — Agentic Multimodal RAG

Status: Draft v0.7 · **Date:** 2026-09-29 · **Implements:** [02-technical-specification.md](#) v0.9 · **Governed by:** [01-constitution.md](#) · **Diagram:** [06-lld-diagram.pdf](#)

The high-level design (what the components are and how data flows) is in spec §3 and [04-flow-diagram.pdf](#). This document is the **low-level design**: modules, functions and their contracts, database tables, algorithms, error handling and tests. It is what a developer implements from.

It contains no code. Function names and signatures are design contracts and may be refined during implementation, as long as the behaviour described here is kept.

1. Module structure

Package / module	Responsibility	May depend on
<code>mmrag/config.py</code>	Load <code>config.yaml</code> + <code>.env</code> , validate, expose typed settings	–
<code>mmrag/ingest/parse.py</code>	PDF → elements, figure PNGs, tables, textless pages	config
<code>mmrag/ingest/bundle.py</code>	Build the GPU captioning bundle	config, parse outputs
<code>gpu_job/caption.ipynb</code>	Run the VLM on the bundle (Kaggle/Colab)	standalone, no <code>mmrag</code> import
<code>mmrag/ingest/enrich.py</code>	Import captions, cross-check, link figures, summarize tables	config, llm client
<code>mmrag/index/chunk.py</code>	Build text, figure and table search documents	config
<code>mmrag/index/embed.py</code>	Embeddings with batching, cache and retries	config, llm client
<code>mmrag/index/writer.py</code>	Transactional write of one document to PostgreSQL	db
<code>mmrag/db/</code>	Connection pool, migrations runner, SQL queries	config
<code>mmrag/retrieval/hybrid.py</code>	Hybrid search (pgvector + full-text + RRF), optional rerank	db, embed
<code>mmrag/agent/graph.py</code>	LangGraph <code>StateGraph</code> : state, nodes (plan, agent, tools, compose, validate, repair), edges, checkpointers	llm client, tools, validator
<code>mmrag/memory/</code>	Long-term memory (Phase 9): extract semantic statements and episode summaries, store, recall	LangGraph store, llm client, embed
<code>mmrag/agent/tools.py</code>	Tool schemas, dispatcher, tool implementations	retrieval, db, charts
<code>mmrag/agent/validator.py</code>	Answer validation, citation hydration, repair	db
<code>mmrag/charts/engine.py</code>	Chart validation and rendering	–
<code>mmrag/llm.py</code>	Thin OpenAI client wrapper (P6): chat, embeddings, retries, usage accounting	config
<code>mmrag/obs/telemetry.py</code>	OpenTelemetry setup: tracer, meter, JSON logging with trace IDs, exporters; span and attribute helpers	config
<code>mmrag/obs/metrics.py</code>	Metric instruments (§5.9)	telemetry

Package / module	Responsibility	May depend on
<code>mmrag/obs/querylog.py</code>	Writes <code>query_log</code> rows (with <code>trace_id</code>); retention cleanup	db, telemetry
<code>eval/dataset.py</code>	Load and validate the golden set; RAGAS test-set drafting	config
<code>eval/runner.py</code>	Run the agent on each golden question; build RAGAS samples; run RAGAS and custom metrics; store results; regression gate	agent, obs, db
<code>eval/custom_metrics.py</code>	Chart numeric correctness, figure hit rate, citation accuracy, refusal	–
<code>eval/report.py</code>	HTML/Markdown report with per-metric, per-question-type and worst-question views, linked to traces	db
<code>mmrag/cli.py</code>	Command-line entry points	all of the above
<code>mmrag/auth/passwords.py</code>	Argon2id hash, verify and rehash; password policy check	config
<code>mmrag/auth/sessions.py</code>	Create, validate, touch and revoke sessions (token hashes in the DB)	db, config
<code>mmrag/auth/service.py</code>	<code>login</code> , <code>logout</code> , <code>change_password</code> , <code>require_user</code> , <code>check_limits</code> , lockout, audit events	passwords, sessions, db
<code>app/login.py</code>	Sign-in and change-password screens	auth
<code>app/ui.py</code>	Streamlit UI	agent, auth, db

Dependency rule: dependencies point downwards (UI → auth / agent → retrieval/charts → db/llm → config). No module imports the UI, and `gpu_job` shares no code with `mmrag`, so it runs on a bare GPU notebook.

`mmrag/obs` is the one cross-cutting exception: every module may call it to create spans, metrics and log lines. It depends only on config (and the database, for `querylog`).

2. Configuration

All tunable values live in `config.yaml`. Secrets live only in `.env`.

Key	Default	Used by
<code>paths.data_dir</code>	<code>data/</code>	all ingest modules
<code>parse.render_dpi</code>	200	figure and page rendering
<code>parse.min_image_px</code>	50	decorative-image filter
<code>parse.textless_chars</code>	50	textless-page detection
<code>parse.furniture_width_ratio</code>	0.85	header/footer rule filter
<code>parse.line_aspect_ratio</code>	40	thin-line filter
<code>parse.cluster_merge_distance_pt</code>	tuned in Phase 1	vector-figure clustering
<code>parse.min_cluster_shapes</code>	tuned in Phase 1	vector-figure clustering

Key	Default	Used by
caption.model_path	awq-7b nf4-7b 3b	GPU job
caption.max_pixels	≈ 1.0 M (1280×28×28)	GPU job
caption.prompt_version	v1	GPU job, cache key
enrich.label_overlap_min (τ)	0.5	label cross-check
enrich.proximity_window_pt	150	figure linking fallback
chunk.min_tokens / max_tokens	400 / 800	chunking
chunk.overlap	0.15	chunking
embed.model / embed.dims	text-embedding-3-large / 1536	embed, schema
embed.batch_size	100	embed
search.candidates	30	hybrid search
search.rrf_k	60	hybrid search
search.top_k	8	hybrid search
search.hnsw_ef_search	64	hybrid search
search.rerank	false	hybrid search (Phase 6)
agent.model / agent.effort	set in Phase 0 (D4)	agent
agent.rounds_default / rounds_multi	3 / 5	agent
agent.summary_model	a low-cost OpenAI model	table summaries
ui.page_dpi	150	source highlighting
auth.min_password_length	12	password policy
auth.common_passwords_file	data/security/common-passwords.txt	password policy
auth.max_failed_attempts / auth.lockout_minutes	5 / 15	account lockout
auth.ip_max_failed_attempts	20 per 15 minutes	per-IP throttle
auth.session_idle_hours / auth.session_absolute_days	8 / 7	sessions
auth.daily_question_limit / auth.daily_cost_limit_usd	100 / 2.00	per-user limits
auth.limits_timezone	local time zone	when "today" starts
auth.trust_proxy_headers	false (true only behind the HTTPS proxy)	client IP
observability.enabled	true	telemetry on/off
observability.service_name	mmrag-app / mmrag-ingest	trace resource

Key	Default	Used by
<code>observability.otlp_endpoint</code>	Phoenix's OTLP endpoint (docker-compose)	exporter
<code>observability.capture_content</code>	true (dev) / false (prod)	prompts, tool args, responses in spans
<code>observability.sample_ratio</code>	1.0 (production sampling done in the Collector)	tracer
<code>observability.log_level</code>	INFO	logging
<code>observability.log_file</code>	<code>data/logs/app.jsonl</code> , rotated daily, 30 files kept	logging
<code>retention.query_log_days</code> / <code>retention.auth_events_days</code>	90 / 365	<code>obs cleanup</code>
<code>eval.golden_set</code>	<code>eval/golden_set.jsonl</code>	evaluation
<code>eval.judge_model</code>	set in Phase 6 (D14)	RAGAS judge
<code>eval.metrics</code>	the list in spec §9	evaluation
<code>eval.regression_tolerance</code>	0.03 (per RAGAS metric)	regression gate
<code>memory.enabled</code> / <code>memory.retention_days</code>	true / 365 (Phase 9)	long-term memory (§5.11); users can switch it off
<code>memory.recall_k</code>	5 (Phase 9)	memories passed to the planner

Secrets (`.env`): `OPENAI_API_KEY` , `DATABASE_URL` .

Validation: config is checked at startup. An unknown key, a wrong type, or `embed.dims` above 2,000 stops the program with a clear message.

3. Ingestion path

3.1 Identifiers

ID	Format	Stability
<code>doc_id</code>	First 16 hex characters of SHA-256(file bytes)	Changes when the PDF content changes
<code>element_id</code>	<code>{doc_id}:p{page}:{type}:{n}</code> , where <i>n</i> is the order on the page	Stable for the same PDF content
<code>chunk_id</code>	<code>{doc_id}:{collection}:{n}</code>	Stable for the same content + chunking config
<code>content_hash</code>	SHA-256 of element text or image bytes	Cache key

Citation IDs: the agent cites a single `id` .

- An `element_id` (figures, tables) hydrates to one location.

- A `chunk_id` (text passages) hydrates to one location per element in its `element_ids`, in reading order.

No other citation fields are ever accepted from the model.

3.2 `ingest/parse.py`

Function	Input → output	Behaviour
<code>profile_pdf(path)</code>	PDF → profile record	Page count, images, vector-heavy pages, tables and textless pages, per page
<code>extract_text_blocks(page)</code>	page → text elements	Blocks in reading order with bbox. Headings: a block whose font size is above the page's body-text size (median) by a set margin, or bold and short. Builds <code>section_path</code> . Headers/footers: text repeated at the same vertical position on more than half the pages is removed and logged.
<code>extract_images(page)</code>	page → image elements + PNGs	Native resolution. De-duplicated by SHA-256. Images under <code>min_image_px</code> are marked <code>skipped: decorative</code> .
<code>detect_tables(page)</code>	page → table elements	Runs before figure detection. Low-confidence tables are also rendered as an image for VLM transcription.
<code>detect_vector_figures(page, table_bboxes)</code>	page → figure elements + PNGs	The 5-step filter from spec §6.1: (1) mask table regions; (2) drop page furniture; (3) drop isolated simple shapes and text-only callouts; (4) keep box-and-arrow clusters; (5) cluster, merge, include labels, add a margin, render at <code>render_dpi</code> . Rejected regions are recorded for the review sheet.
<code>detect_textless_pages(doc)</code>	doc → scanned-page elements	Pages under <code>textless_chars</code> are rendered whole and queued for the VLM
<code>write_outputs(doc)</code>	→ <code>data/elements/</code> , <code>data/tables/</code> , <code>data/assets/</code> , <code>skip_log.jsonl</code> , <code>review_sheet.html</code>	Audit files (P8)

Invariant: every page produces at least one element or a `skip_log` entry (P1).

3.3 `ingest/bundle.py`

`build_jobs(doc)` creates one job per figure and per textless page, containing: `element_id`, image path, page heading, about 150 words of text before and after, and any "Figure N" reference. The jobs are written to `jobs.jsonl`, and zipped together with the PNGs into `bundle.zip`. A job is left out if a caption already exists in the cache for (image hash, model path, prompt version).

3.4 `gpu_job/caption.ipynb` (runs on Kaggle/Colab)

1. `load_model(model_path)`: AWQ-7B on vLLM (fp16), NF4-7B on `transformers` + `bitsandbytes`, or 3B on `transformers` (fp16).

2. The image processor applies `max_pixels`.
3. For each batch: generate. The output is parsed as JSON and validated against the caption schema (spec §5.2) with pydantic.
 - Invalid output → JSON repair → retry once → otherwise the record is written with `status: needs_review`.
4. After each batch, the processed IDs are appended to `done_ids.txt`. On restart, done jobs are skipped (resumable).
5. Output: `captions.jsonl`, plus a run report (model path, GPU memory peak, speed, validity rate).

3.5 ingest/enrich.py

Function	Behaviour
<code>validate_captions()</code>	Re-validates every caption against the schema on the laptop. Invalid or <code>confidence: low</code> → <code>needs_review</code> .
<code>cross_check_labels()</code>	For vector figures: the share of the VLM's <code>visible_text</code> tokens found in the PDF text inside the figure bbox. Below τ → <code>needs_review</code> .
<code>link_elements()</code>	For each figure and table , the first rule that produces a link wins: (1) explicit "Figure/Fig./Table N"; (2) deictic phrase ("below/above/shown here") pointing at the nearest figure or table in that direction; (3) most related nearby paragraph : candidates within <code>proximity_window_pt</code> on the same page plus the paragraphs directly before and after, scored <code>cosine(embedding) + label_weight × label share</code> ; the best one at or above <code>link_min_score</code> is linked, else the item is reported unlinked. Writes <code>element_links</code> with <code>method</code> and <code>score</code> .
<code>summarize_tables()</code>	Calls <code>agent.summary_model</code> for a 2-sentence summary of each table's title, headers and rows. Cached by (table <code>content_hash</code> , model). A comparison mode runs several models side by side for the owner to choose.

3.6 index/chunk.py

Function	Behaviour
<code>chunk_text(doc)</code>	Joins consecutive text blocks within one section until <code>max_tokens</code> , with <code>overlap</code> carried into the next chunk. A chunk never crosses a section boundary. A chunk shorter than <code>min_tokens</code> is only allowed at a section end. Each chunk is prefixed with its <code>section_path</code> . Records <code>element_ids[]</code> .
<code>build_figure_docs(doc)</code>	One document per figure: <code>short_caption</code> + <code>detailed_description</code> + <code>visible_text</code> + <code>linked paragraph</code> . The same text goes to <code>dense_text</code> and <code>keyword_text</code> .
<code>build_table_docs(doc)</code>	<code>dense_text</code> = summary + title + headers. <code>keyword_text</code> = title + headers + all cell values.

3.7 index/embed.py

- `embed_batch(texts)` sends at most `batch_size` texts per request, using `embed.model` and `embed.dims`.
- Cache key: (model, dims, SHA-256(text)), stored under `data/cache/embeddings/`. Only cache misses are sent.
- Retries: exponential backoff, 3 attempts, on rate limits (429), server errors (5xx) and timeouts. Other errors fail the document; nothing has been written to the database at that point.

3.8 index/writer.py — write_document(doc)

1. If a `documents` row with the same `content_hash` is already `active` → **no-op** (idempotent).
2. `BEGIN` a transaction and take an advisory lock on `doc_id`, so two runs can't ingest the same PDF at once.
3. Insert the new `documents` row with `version = previous + 1` and `status = active`.
4. Insert all `elements`, `figure_captions`, `doc_tables`, `figure_links` and `search_chunks`. The `tsvector` columns are generated by the database.
5. Delete the previous version's `documents` row; foreign keys cascade to all of its rows.
6. `COMMIT`. On any error, `ROLLBACK`: the previous version stays fully searchable (NFR-9).

All embeddings are computed **before** step 2, so the transaction holds no network calls and stays short.

3.9 cli.py

Command	Behaviour
<code>ingest add <pdf></code>	parse → bundle → (GPU job run manually, then <code>captions.jsonl</code> copied back) → enrich → chunk → embed → write
<code>ingest replace <pdf></code>	Same as <code>add</code> ; the old version is swapped out in the same transaction (§3.8 step 5)
<code>ingest remove <pdf></code>	Deletes the <code>documents</code> row (cascade) and moves its assets to an archive folder
<code>db migrate</code>	Applies <code>db/migrations/*.sql</code> in order, recording applied versions in a migrations table
<code>ask "<question>"</code>	Runs the agent and writes the answer to an HTML file (Phase 4)
<code>user add <email> --role user</code> (or <code>admin</code>)	Creates the account with a random temporary password, shown once; <code>must_change_password = true</code>
<code>user reset-password <email></code>	New temporary password, shown once; revokes all of the user's sessions
<code>user disable <email></code> / <code>user enable <email></code>	A disabled user cannot sign in; their sessions are revoked
<code>user unlock <email></code>	Clears a lockout
<code>user list</code>	Email, role, status, last sign-in
<code>obs cleanup</code>	Deletes <code>query_log</code> and <code>auth_events</code> rows older than their retention period
<code>eval draft-questions</code>	Drafts candidate questions with RAGAS's test-set generator into <code>eval/drafts.jsonl</code> for owner review; never writes to the golden set
<code>eval run [--baseline]</code>	Runs the full evaluation (§5.10); <code>--baseline</code> stores the run as the new baseline; exits non-zero on regression
<code>eval report <run_id></code>	Writes the report for a run

4. Database design (PostgreSQL + pgvector)

4.1 Tables

documents

Column	Type	Constraints
doc_id	text	PK
source_file	text	not null
content_hash	text	not null
version	int	not null
status	text	active . Old versions are deleted on replace (§3.8), not kept; other states are reserved.
ingested_at	timestampz	default now

elements

Column	Type	Constraints
element_id	text	PK
doc_id	text	FK → documents, on delete cascade
page	int	≥ 1
bbox	float8[4]	x0, y0, x1, y1 in PDF points
type	text	text · image · vector_figure · table · scanned_page
section_path	text[]	
text	text	nullable
asset_path	text	nullable; relative to data/assets/
content_hash	text	
status / skip_reason	text	ok · skipped · needs_review

figure_captions

Column	Type	Notes
element_id	text	PK, FK → elements (cascade)
figure_type	text	enum from spec §5.2
short_caption	text	
detailed_description	text	
visible_text	text[]	
extracted_data	jsonb	chart values with an exact / estimated flag per value
keywords	text[]	

Column	Type	Notes
confidence	text	
model_id, prompt_version	text	reproducibility

doc_tables

Column	Type	Notes
element_id	text	PK, FK → elements (cascade)
columns, rows, units	jsonb	kept exactly as extracted
numeric_columns	text[]	
title, summary	text	

element_links (was `figure_links`; tables are linked too since v0.7)

Column	Type	Notes
target_id	text	the figure or table; FK → elements (cascade)
text_element_id	text	FK → elements (cascade)
method	text	explicit · deictic · related
score	real	1.0 for explicit and deictic; the similarity score for related

search_chunks

Column	Type	Notes
chunk_id	text	PK
doc_id	text	FK → documents (cascade)
collection	text	text · figure · table
element_ids	text[]	elements the chunk covers
dense_text	text	embedded
keyword_text	text	full-text indexed
embedding	vector(1536)	dimension = <code>embed.dims</code>
embedding_model	text	for blue-green switches
tsv_english	tsvector	generated from <code>keyword_text</code> with the <code>english</code> config
tsv_simple	tsvector	generated from <code>keyword_text</code> with the <code>simple</code> config

query_log

Column	Type
query_id	uuid, PK

Column	Type
user_id	uuid, FK → users
trace_id	text (32 hex characters; links to the OpenTelemetry trace)
question	text
qtype	text
rounds	int
tool_calls	jsonb
tokens_in, tokens_out	int
cost_usd	numeric
latency_ms	int
validator_result	text
answer_json	jsonb (hydrated answer, used to restore chat history)
created_at	timestampz

users

Column	Type	Constraints
user_id	uuid	PK
email	text	unique; stored lower-case
password_hash	text	Argon2id encoded hash (includes salt and parameters)
role	text	admin · user
status	text	active · disabled
must_change_password	boolean	
failed_attempts	int	reset on successful sign-in
locked_until	timestampz	nullable
created_at, last_login_at	timestampz	

sessions

Column	Type	Constraints
session_id	uuid	PK
user_id	uuid	FK → users, on delete cascade
token_hash	text	SHA-256 of the session token; unique
created_at, last_seen_at, expires_at	timestampz	
revoked_at	timestampz	nullable

auth_events

Column	Type	Constraints
event_id	bigint identity	PK
user_id	uuid	nullable FK → users
email_attempted	text	lower-case
event_type	text	login_success · login_failure · lockout · logout · password_change · password_reset · user_created · user_disabled · user_enabled
ip	inet	nullable
created_at	timestampz	

eval_runs

Column	Type	Notes
run_id	uuid	PK
started_at, finished_at	timestampz	
git_commit, config_hash	text	what was evaluated
agent_model, judge_model, ragas_version, golden_set_version	text	reproducibility
is_baseline	boolean	
summary	jsonb	mean score per metric and per question type
passed	boolean	regression gate result
trace_id	text	the eval.run trace

eval_results

Column	Type	Notes
run_id	uuid	FK → eval_runs (cascade)
question_id	text	from the golden set
metric	text	e.g. faithfulness, chart_numeric
score	numeric	0–1
details	jsonb	judge reasoning or failed values
trace_id	text	the question's trace

Primary key: (run_id, question_id, metric).

schema_migrations: version (PK), applied_at.

4.2 Local database setup (dev)

- **Image:** official `pgvector/pgvector:pg16` Docker image, defined in `docker-compose.yml`.
- **Data:** stored in a **named Docker volume**. Never bind-mount a Windows folder as the data directory: Postgres refuses to start when it can't set Linux permissions on NTFS.
- **Port:** a host port other than 5432 if the existing local Postgres already uses it; the URL lives in `DATABASE_URL` in `.env`.
- **Backups:** `pg_dump` / `pg_restore`, not copying the volume.
- **Production:** managed Postgres with pgvector; same migrations.

4.3 Indexes

Index	Purpose
HNSW on <code>search_chunks.embedding</code> (cosine), one partial index per collection	Semantic search that still returns k results after the collection filter
GIN on <code>tsv_english</code> and on <code>tsv_simple</code>	Keyword search
B-tree on <code>(collection, doc_id)</code> and on <code>elements(doc_id, page)</code>	Filters and page lookups
B-tree on <code>element_links(target_id)</code> and <code>element_links(text_element_id)</code>	Links in both directions (related items, §5.4)
Unique on <code>users(email)</code> and on <code>sessions(token_hash)</code>	Sign-in and session lookups
B-tree on <code>query_log(user_id, created_at)</code>	Per-user limits and history
B-tree on <code>auth_events(email_attempted, created_at)</code> and <code>auth_events(ip, created_at)</code>	Lockout and per-IP throttling

5. Query path

5.1 Request lifecycle

The whole request runs inside **one OpenTelemetry trace** (§5.9); each step below is a span.

1. `app/ui.py` → `auth.require_user()` (§5.8) → `on_submit(q)` → `auth.check_limits(user)` → `agent.graph.run_query(q, user, thread_id)`.
2. **Plan:** the planner classifies `qtype` (conceptual · visual · quantitative · mixed · multi-part) and sets `round_limit` (3, or 5 for multi-part).
3. **Tool rounds:** each LLM turn may emit several tool calls. The dispatcher runs them in parallel and returns every result in one message.
4. **Compose:** the model returns the Answer JSON (spec §5.4), citing by `id` only (§3.1 Citation IDs).
5. **Validate:** `validator.validate(answer, evidence)` hydrates the citations. On failure: one repair turn.
6. **Log** to `query_log` with `user_id` and `answer_json`; return the Answer to the UI for rendering.

5.2 `agent/graph.py` (LangGraph, D3)

State (a typed dict carried between nodes and checkpointed per `thread_id`): `messages` (the conversation, including tool calls and results), `question`, `qtype`, `round`, `round_limit`, `ledger` (evidence by id), `memories`

(Phase 9), `answer`, `validation`, `repaired`.

Nodes and edges:

Node	Does	Next
<code>recall_memory</code>	(Phase 9) the user's top memories by meaning, as user context	<code>plan</code>
<code>plan</code>	classify <code>qtype</code> , set <code>round_limit</code>	<code>agent</code>
<code>agent</code>	one OpenAI call with the tool schemas; may return several tool calls	<code>tools</code> if tool calls and <code>round < round_limit</code> , else <code>compose</code>
<code>tools</code>	run the calls in parallel (thread pool, per-call timeout), add results to <code>messages</code> and <code>ledger</code>	<code>agent</code>
<code>compose</code>	OpenAI structured output: the Answer JSON (spec §5.4); at the round limit, told to state what is missing	<code>validate</code>
<code>validate</code>	§5.6; hydrates citations	END if ok, <code>repair</code> if failed and not yet repaired, else END with failing blocks dropped and a notice
<code>repair</code>	one compose turn given the validator's failures	<code>validate</code>
<code>remember</code>	(Phase 9) extract memories after a validated answer	END

Rules: nodes call the OpenAI SDK directly (no LangChain chat models, no prebuilt agents); tools are plain functions from `agent/tools.py`; each node is also an OpenTelemetry span (§5.9); the LangGraph version is pinned. The checkpointer is LangGraph's Postgres saver on the project database; `thread_id` is recorded in `query_log`.

Carried over from the loop design:

- **System prompt**, a versioned file, contains:
 - the grounding rules (P5);
 - the citation rules (ids only);
 - the chart rules (P4);
 - the block format;
 - the rule "if not found, say so".
- **Evidence ledger:** every tool result is stored by ID (chunk, element, table, compute result). The validator checks answers against this ledger.
- **Termination:** the loop stops when the model makes no tool call (then it composes), or when `rounds == round_limit`. At the limit, a final compose turn is forced with the instruction to state what is missing.
- **Structured output:** the compose turn requires the Answer JSON schema. A response that doesn't parse gets one retry.
- **Streaming:** the graph streams node and tool events; the UI shows progress ("searching figures...").

5.3 `agent/tools.py`

Tools are in-process Python functions; no MCP server (D16).

Tool	Parameters	Returns	Errors (returned to the model, never raised)
<code>search_text</code>	<code>query: str, k: int ≤ 20</code>	<code>[[chunk_id, text, section_path, source_file, page]]</code>	empty result is not an error
<code>search_figures</code>	<code>query, k</code>	<code>[[element_id, short_caption, description_excerpt, source_file, page]]</code>	–
<code>search_tables</code>	<code>query, k</code>	<code>[[element_id, title, summary, columns]]</code>	–
<code>get_figure</code>	<code>element_id</code>	full caption record + image (for the model to view)	unknown id
<code>get_table</code>	<code>element_id</code>	<code>{columns, rows, units, title}</code>	unknown id
<code>view_page</code>	<code>source_file, page</code>	page PNG at <code>ui.page_dpi</code>	unknown file or page
<code>compute</code>	<code>expression, refs: {name: evidence_id}</code>	number + provenance	disallowed syntax; ref not in ledger
<code>make_chart</code>	<code>chart_type, title, labels, series[{name, values, unit}], value_refs, citations</code>	<code>{chart_id, png_path, spec}</code>	failed validation, with the reason

Dispatcher:

- Validates the arguments against each tool's JSON schema.
- Runs calls on a thread pool.
- Applies a per-call timeout.
- Turns any exception into an error `tool_result` carrying the message, so the model can recover.

`compute` parses the expression into a syntax tree and allows only numbers, named refs, `+` `-` `*` `÷`, parentheses, `round`, `sum`, `min`, `max` and percentages. No `eval`, no names outside `refs`.

5.4 retrieval/hybrid.py — `search(query, collection, k)`

1. `q_vec = embed_query(query)`, cached in an in-process LRU cache by query text.
2. **One SQL statement** with three parts (CTEs):
 - **semantic**: the candidates nearest `search_chunks` rows by cosine distance, restricted to `collection`, with `hnsw.ef_search` set for the query. Each row gets its rank (1...*n*).
 - **keyword**: rows where `tsv_english` matches `websearch_to_tsquery('english', query)` **or** `tsv_simple` matches `websearch_to_tsquery('simple', query)`. Ranked by the higher `ts_rank_cd` of the two, limited to `candidates`.
 - **rff**: a full outer join of both lists on `chunk_id`. Score = $\sum 1/(rff_k + rank)$, where a missing rank contributes 0. Ordered by score, limited to *k*.
3. **Location aggregation**: for each of the top *k* chunks, a lateral subquery collects its elements into **one JSON array** of `{element_id, page, bbox}`. The array is ordered by each element's position in `element_ids` (reading order), and `source_file` comes from `documents`. Each chunk stays **exactly one**

row. A plain join on `element_id = ANY(element_ids)` would return one row per element, duplicating chunks and breaking the k limit.

- If `search.rerank` is on, the top 20 are re-scored with the local cross-encoder before cutting to k .
- Related items:** one follow-up query on `element_links` (both directions) for the elements of the top k chunks; each result gets `related: [{element_id, type, page, title}]` (a figure's short caption, a table's title, a paragraph's first words).
- All values are passed as SQL parameters, never string-built.

5.5 `charts/engine.py`

- `validate(spec, ledger)`:
 - every value must equal a number in the evidence ledger or a `compute` result, after normalization: thousands separators, $\% \leftrightarrow$ fraction, units;
 - labels and value counts must match;
 - one unit per axis;
 - pie charts need non-negative values that form parts of a whole (≤ 8 slices), otherwise they become bar charts;
 - any `estimated` value sets `approximate = true`.
- `render(spec)`: a Plotly figure (interactive in the UI) plus a PNG, and the data table to show under the chart.

5.6 `agent/validator.py` — `validate(answer, ledger)`

Check	Failure action
Answer JSON matches the block schema	repair turn
Every block has ≥ 1 citation; each ID exists in the DB and in the ledger	repair turn
Citation hydration: <code>element_id</code> $\rightarrow$ file, page, bbox from <code>elements</code> ; <code>chunk_id</code> $\rightarrow$ its <code>element_ids</code> $\rightarrow$ one (page, bbox) pair each, in reading order	– (always done)
<code>image</code> blocks: <code>asset_path</code> is taken from the DB, never from the model	–
<code>chart</code> blocks: chart-engine validation passes	repair turn
Numbers in <code>text</code> blocks that look like data points are found in the ledger	flagged, then repair turn

Repair policy:

- One repair turn, which gets the list of failures.
- If it still fails, the failing blocks are **dropped** and a visible notice is added: "Some content was removed because it could not be verified against the documents."
- An answer is never shown with an unverified block (P2, P4).

5.7 `app/ui.py`

State: Streamlit re-runs the whole script on every interaction.

- Each finished turn is stored in `st.session_state["chat_history"]`: question, hydrated Answer blocks, Plotly figures and chart data tables.

- The chat view always renders from this state. Clicking a citation, expanding a table or opening a source **never** re-runs the agent, validator, chart engine or database search.
- Rendered page images are cached by (file, page, DPI) with `st.cache_data`, and the database connection with `st.cache_resource`.
- Only a new question triggers `run_query`.
- `render_blocks(answer)`:
 - **text**: markdown with citation chips;
 - **image**: original PNG with its caption;
 - **chart**: Plotly chart with a collapsible data table and an "approximate" badge when flagged;
 - **table**: the table itself;
 - **sources**: a list of the cited sources.
- `show_source(citation)`:
 1. Render the page at `ui.page_dpi` (rotation applied).
 2. Convert each bbox from points to pixels ($\times \text{DPI}/72$).
 3. Draw semi-transparent rectangles over the page.

5.8 Authentication (`mmrag/auth/`, `app/login.py`)

Sign-in — `login(email, password, ip)`:

1. Normalize the email (trim, lower-case).
2. **Per-IP throttle**: if the failures from this IP within the window reach `ip_max_failed_attempts`, reject with the generic message.
3. Load the user. If the user is missing or disabled: verify against a fixed dummy hash (equal timing), record `login_failure`, and return the generic message.
4. If `locked_until` is in the future: record `login_failure` and return the generic message.
5. Verify the Argon2id hash.
 - On failure: `failed_attempts += 1`. When it reaches `max_failed_attempts`, set `locked_until = now + lockout_minutes` and record `lockout`. Return the generic message.
6. On success:
 - reset `failed_attempts`, and rehash if the hash parameters are outdated;
 - create a session: a random 32-byte token, whose SHA-256 hash is stored with its idle and absolute expiry;
 - record `login_success` and set `last_login_at`;
 - put the token in `st.session_state`.
7. If `must_change_password` is set, show the change-password screen before the chat.

Every rerun — `require_user()`:

- Hash the token from `st.session_state` and load its session.
- The session is valid only if it is not revoked, is within its absolute expiry and idle limit, and belongs to an active user.
- Valid: update `last_seen_at` (at most once a minute) and return the user.
- Invalid: clear the session state and show the sign-in screen.

Before each question — `check_limits(user)` :

- Sum today's questions and `cost_usd` from `query_log` for the user (`auth.limits_timezone`).
- Over either limit: show a message with the reset time, and do not run the agent.

Logout, password change, reset, disable:

- Set `revoked_at` on all of the user's sessions and record the event.
- A password change requires the current password; the new one must pass the policy and differ from the current one.

Chat history: loaded from `query_log.answer_json` for the signed-in user only, into `st.session_state["chat_history"]`.

5.9 Observability (`mmrag/obs/`)

Setup — `init_telemetry(service_name)`, called once at startup by `cli.py` and `app/ui.py` (in Streamlit, guarded with `st.cache_resource` so reruns don't initialize it again):

- **Tracer provider:** resource attributes `service.name`, `service.version` and `deployment.environment`. Spans go through a **batch span processor** to an OTLP exporter, which is non-blocking with a bounded queue.
- **Auto-instrumentation:** the OpenAI SDK (OpenTelemetry GenAI instrumentation) and `psycopg` (SQL spans; statement text only, never parameter values).
- **Logging:** Python `logging` with a JSON formatter.
 - Every record carries `trace_id`, `span_id`, level, module and message.
 - Output goes to standard output and to `observability.log_file`.
- **Meter provider** for the metrics below, exported over OTLP.

Span tree for one question:

Span	Parent	Key attributes
<code>query</code>	root	<code>mmrag.request_id</code> (= trace id), <code>user.id</code> , <code>mmrag.qtype</code> , <code>mmrag.round_limit</code> , <code>mmrag.rounds_used</code> , <code>mmrag.validator_result</code> , <code>mmrag.cost_usd</code>
<code>auth.check_limits</code>	query	<code>mmrag.questions_today</code> , <code>mmrag.cost_today_usd</code> , <code>mmrag.limit_hit</code>
<code>agent.plan</code>	query	<code>mmrag.qtype</code>
<code>agent.round</code> (one per round)	query	<code>mmrag.round</code> , <code>mmrag.tool_call_count</code>
<code>chat <model></code> (auto)	plan / round / compose	<code>gen_ai.system</code> , <code>gen_ai.request.model</code> , <code>gen_ai.usage.input_tokens</code> , <code>gen_ai.usage.output_tokens</code> , <code>mmrag.cost_usd</code>
<code>tool.<name></code>	round	<code>mmrag.tool.name</code> , <code>mmrag.tool.result_count</code> , error status; arguments only if content capture is on
<code>retrieval.hybrid_search</code>	<code>tool.search_*</code>	<code>mmrag.collection</code> , <code>mmrag.k</code> , <code>mmrag.semantic_hits</code> , <code>mmrag.keyword_hits</code>

Span	Parent	Key attributes
<code>embeddings <model> (auto)</code>	<code>hybrid_search</code>	<code>model</code> , <code>input tokens</code>
SQL query (auto)	<code>hybrid_search / tools</code>	<code>db.system = postgresql</code> , statement without values
<code>charts.render</code>	<code>tool.make_chart</code>	<code>mmrag.chart_type</code> , <code>mmrag.approximate</code>
<code>agent.compose</code>	<code>query</code>	–
<code>validator.validate</code>	<code>query</code>	<code>mmrag.validator_result</code> , <code>mmrag.failures</code>
<code>agent.repair</code>	<code>query</code>	only when a repair runs

Other traces:

- **Ingestion:** `ingest.document` (`doc_id`, `source_file`, `pages`) → `ingest.parse` → `ingest.bundle` → `ingest.enrich` (with LLM spans for table summaries) → `index.chunk` → `index.embed` (batches, cache hits) → `index.write_document` (rows written, committed or rolled back). The GPU notebook can't reach the trace viewer, so its run report is attached as attributes when captions are imported.
- **Sign-in:** an `auth.login` span records only the outcome and `user.id` when known. It never records the email or the password.

Metrics:

Instrument	Type	Attributes
<code>mmrag.questions</code>	counter	<code>qtype</code> , <code>outcome</code>
<code>mmrag.question.duration</code>	histogram (s)	<code>qtype</code>
<code>mmrag.llm.tokens</code>	counter	<code>model</code> , <code>direction</code> (input/output)
<code>mmrag.llm.cost_usd</code>	counter	<code>model</code>
<code>mmrag.tool.calls</code>	counter	<code>tool</code> , <code>outcome</code>
<code>mmrag.validator.failures</code>	counter	<code>reason</code>
<code>mmrag.auth.failures / mmrag.auth.lockouts</code>	counter	–
<code>mmrag.ingest.stage.duration</code>	histogram (s)	<code>stage</code>

Content capture:

- **When `observability.capture_content` is on (dev):** prompts, tool arguments and model responses are added as span events.
- **When it's off (prod):** only sizes, counts and IDs are recorded.
- **Never recorded, in either mode:** API keys, passwords, session tokens, email addresses.
- **Naming:** GenAI attribute names are defined in one module, because the OpenTelemetry GenAI conventions are still evolving.

Correlation: `query_log.trace_id` = the question's trace ID, and every log line carries it. The admin page links each answer to its trace in the viewer.

Export:

- **Dev:** OTLP straight to Phoenix (`docker-compose.yml`).
- **Prod:** OTLP to an OpenTelemetry Collector. It batches data and tail-samples it (keeping every error and slow trace) before forwarding to the chosen backend.
- Switching backends is a config change.

Retention:

- Traces: 14 days (a viewer setting).
- App log files: 30 days (rotation).
- `query_log`: 90 days, and `auth_events`: 365 days, enforced by `obs cleanup`.

Failure safety: telemetry code never raises into application code. Exporter failures drop data and log a warning at most once a minute. The app keeps answering with the viewer stopped.

5.10 Evaluation (RAGAS + custom metrics)

Golden set (`eval/golden_set.jsonl`, versioned). Each line holds:

- `question_id`, `question`, `qtype`;
- `reference_answer` and `reference_ids` (element/chunk IDs);
- `expected_figure_ids`, `expected_chart_values`, `expected_tool_calls`;
- `answerable`.

The file is validated on load.

`eval run` steps:

1. Create an `eval_runs` row and an `eval.run` trace.
2. For each question, run `run_query` with the production configuration, as an evaluation user whose daily limits don't apply. Collect:
 - the answer blocks;
 - the evidence ledger: retrieved text chunks, figure captions and images, tables;
 - the tool calls made;
 - the question's `trace_id`.
3. **Build RAGAS samples:**
 - **single-turn:** question, answer text, retrieved text contexts, reference answer;
 - **multimodal:** the same, with figure images as contexts;
 - **multi-turn agent:** the message and tool-call sequence, with the expected tool calls.
4. **RAGAS scoring** with `eval.judge_model` (fixed settings) and `embed.model` where a metric needs embeddings. Metrics:
 - Faithfulness, MultiModalFaithfulness, ResponseRelevancy;
 - LLMContextPrecisionWithReference, LLMContextRecall;
 - FactualCorrectness, ToolCallAccuracy.

Class names are checked against the pinned RAGAS version.

5. **Custom metrics** (`custom_metrics.py`):

- **chart numeric:** each charted value equals an expected value after normalization (separators, % vs fraction, units); all-or-nothing per chart.
- **figure hit:** each expected figure ID appears in an image block.
- **citation accuracy:** the share of cited IDs that belong to `reference_ids`.
- **refusal:** for `answerable = false`, the answer must state that the information was not found.
- **latency and cost:** read from `query_log`.

6. Store every score in `eval_results` and the averages in `eval_runs.summary`.

7. **Regression gate:** compared with the latest baseline, the run **fails** (non-zero exit, `passed = false`) if any RAGAS metric drops by more than `eval.regression_tolerance`, or any 100% metric (chart numeric, refusal) is below 100%.

8. **Report:** per-metric averages, a breakdown by question type, and the 5 worst questions with the judge's reasoning and links to their traces.

Judge reliability:

- The judge model differs from the agent model (D14) and stays fixed within a baseline.
- The baseline is run twice to measure variance.
- The owner spot-checks 10 judgments per run.

Cost: each run makes the agent's normal calls plus RAGAS judge calls. It's logged per run from the traces.

5.11 Long-term memory (`mmrag/memory/`, Phase 9)

Function	Behaviour
<code>extract_semantic(user_id, turn)</code>	Low-cost model returns short statements about the user (preferences, focus, expertise), each with a subject key; a statement with an existing key replaces it.
<code>summarize_episode(thread)</code>	At thread end or idle timeout: what was asked, found and left open, in a few sentences.
<code>recall(user_id, question, k)</code>	Semantic search in the user's namespaces (<code>memories/{user_id}/semantic, .../episodic</code>); returns statements with ids and dates.
<code>list / delete / set_enabled</code>	User controls (FR-25); <code>delete_user</code> removes the namespaces.

- **Storage:** LangGraph Store on PostgreSQL with pgvector, using `embed.model` (D15).
- **Use:** recalled memories enter the plan and compose prompts in a section labelled as user context, with the instruction that they are not evidence (P13). The validator rejects citations that are not corpus `element_id / chunk_ids`.
- **Retention:** `memory.retention_days` in config; `obs cleanup` also prunes expired memories.

6. Error handling

Failure	Where	Handling
OpenAI 429 / 5xx / timeout	<code>llm.py</code>	Exponential backoff with jitter, 3 attempts, then a clear error
OpenAI 400 (bad request)	<code>llm.py</code>	No retry; logged with request metadata (no secrets)
Malformed tool arguments	dispatcher	Error <code>tool_result</code> to the model

Failure	Where	Handling
Tool exception / timeout	dispatcher	Error <code>tool_result</code> ; the round still counts
Database connection lost	db/	Pool reconnect with 3 attempts; a query error returns an error <code>tool_result</code>
Ingestion error mid-document	<code>writer.py</code>	ROLLBACK ; the previous version is kept; error in the ingestion log
VLM output invalid	GPU job	Repair → retry → <code>needs_review</code>
Validator failure	validator	One repair turn → drop blocks + notice
Round limit reached	loop	Forced compose that states what is missing
Wrong credentials, locked or disabled account	auth	Same generic message; event recorded in <code>auth_events</code>
Session expired or revoked	<code>require_user</code>	Session state cleared; sign-in screen shown
Daily limit reached	<code>check_limits</code>	Message with reset time; the agent is not run
Trace viewer unreachable	obs	Spans dropped by the batch exporter, with a warning at most once a minute; the app keeps working

7. Caching

Cache	Key	Location	Invalidated by
Captions	image hash + model path + prompt version	<code>data/captions/</code>	changing model or prompt
Embeddings	model + dims + text hash	<code>data/cache/embeddings/</code>	changing model or dims
Table summaries	table content hash + summary model	<code>data/cache/summaries.jsonl</code>	changing the table or model
Query embeddings	query text	in-process LRU	process restart
Whole document	PDF content hash	<code>documents</code> table	new PDF content

8. Logging and observability

Traces, JSON logs and metrics use OpenTelemetry (§5.9). The records below are kept in the database.

- **Query log** (`query_log` table): one row per question, with rounds, tool calls, tokens, cost, latency and validator result. Feeds the Phase 6 evaluation and the UI cost counter.
- **Ingestion log** (`data/logs/ingest.jsonl`): per document and stage, with counts, durations, skips, `needs_review` items and errors.
- **Never logged**: API keys, database passwords, user passwords (plain or hashed), session tokens.
- **Audit log** (`auth_events` table): every sign-in attempt and account change, for security review.

9. Security

- Secrets only in `.env`, which is git-ignored. `.env.example` has placeholders.
- All SQL is parameterized.
- The model never supplies file paths:
 - asset paths come from the database;
 - `view_page` accepts only a `source_file` that exists in `documents`;
 - resolved paths must stay under `data/`, which prevents path traversal.
- `compute` uses a whitelisted syntax tree, never `eval`.
- Tool arguments are validated against their schemas before execution.
- Retrieved PDF text is treated as **data, not instructions**. The system prompt tells the model to ignore instructions found inside documents.
- **Passwords:** Argon2id only. The temporary password from `user add` or `reset-password` is shown once in the CLI and never stored in plain text.
- **Session tokens:** 256-bit random; only hashes stored; revoked on logout, password change, reset and disable.
- **No account enumeration:** the same message and equal timing for every sign-in failure.
- **Authorization:** every screen and every agent run happens only after `require_user()`. Admin actions are CLI-only in v1.
- **Deployment:** HTTPS through a reverse proxy; Streamlit listens only on localhost; `auth.trust_proxy_headers` is enabled only behind that proxy, so client IPs can't be spoofed.
- **Telemetry:** spans and logs never contain secrets, passwords, tokens or email addresses (only `user_id`). Prompt and document content is captured only when `observability.capture_content` is on (dev).

10. Testing

Strategy (constitution W5), a test pyramid:

Part of the system	Approach
Deterministic logic: config, IDs, chunking, RRF, <code>compute</code> , chart validation, validator, citation hydration, bbox conversion, auth, sessions, limits, the transactional writer	Test-driven development: red → green → refactor
Exploratory logic: figure and table detection, heading detection	Tune on the review sheet with the owner, then freeze the approved results as regression tests on fixture pages
LLM behaviour: answer quality, retrieval quality	RAGAS evaluation (\$5.10); unit tests use a mocked OpenAI client
Throwaway experiments	Not tested; deleted afterwards

Tooling:

- `pytest`, with tests in `tests/unit/`, `tests/integration/` and `tests/live/`.
- Markers `unit`, `integration` and `live`. The default run is unit + integration; `live` runs only on request.

- **Test database:** `mmrag_test` in the same Docker Postgres, created and migrated by a session fixture. Each test runs in a transaction that is rolled back.
- **Fixtures:** `tests/fixtures/pages/`, with single pages cut from the PDFs and their expected results as JSON.
- **Mocked OpenAI client:** returns scripted responses and tool calls, so agent-loop tests cost nothing and are repeatable.
- **Coverage:** measured with `pytest-cov` and reported per phase; no fixed percentage target. The test-first rule matters more.
- **CI:** GitHub Actions on every push and pull request (plan Phase 8).

Level	What	Examples
Unit	Pure functions	chunk boundaries; RRF scoring; <code>compute</code> whitelist rejects <code>__import__</code> ; chart validation (pie → bar fallback, value mismatch rejected); bbox point→pixel conversion; ID formats
Component	Module against fixtures	<code>parse.py</code> on 3 sample pages (table grid, box-and-arrow diagram, textless page) with expected element counts; enrich linking rules
Integration	Real PostgreSQL (Docker)	migrations apply cleanly; transactional write with a failure injected mid-document leaves the old version intact; re-ingest is a no-op; cascade delete removes every row; hybrid search returns an exact-term hit via <code>tsv_simple</code> ; each chunk appears exactly once, with locations in reading order; the container keeps its data across restarts
End to end	Full pipeline	Phase 4's 10 sample questions; Phase 6 golden set (40 questions)
Regression	On every change	<code>eval run</code> : RAGAS and custom scores compared with the stored baseline; a regression fails the run (P10)
Security	Authentication	wrong password and unknown email give identical messages; logout after 5 failures and release after 15 minutes; idle and absolute session expiry; logout and password change revoke sessions; a disabled user can't sign in; the limit blocks the 101st question of the day; no password or token appears in any log
Observability	Tracing	one question → one trace with the expected span tree; <code>query_log.trace_id</code> matches it; no content in spans when capture is off; no secrets in any span or log; the app answers normally with the trace viewer stopped

11. Traceability (requirement → design)

Spec requirement	Implemented in
FR-1 text	<code>parse.extract_text_blocks</code>
FR-2 images	<code>parse.extract_images</code>
FR-3 vector diagrams	<code>parse.detect_vector_figures</code>
FR-4 tables	<code>parse.detect_tables</code> , <code>doc_tables</code>
FR-5 textless pages	<code>parse.detect_textless_pages</code> , GPU job
FR-6/7 captions and chart data	GPU job, <code>figure_captions.extracted_data</code>
FR-8 figure linking	<code>enrich.link_figures</code> , <code>figure_links</code>

Spec requirement	Implemented in
FR-9 hybrid index	<code>chunk.py</code> , <code>embed.py</code> , <code>search_chunks</code> + indexes
FR-10 agent	<code>agent/graph.py</code> , <code>agent/tools.py</code>
FR-11 block answers	Answer schema, <code>validator.py</code>
FR-12 truthful charts	<code>charts/engine.py</code> , <code>validator</code>
FR-13 original figures	<code>validator</code> (DB <code>asset_path</code>), <code>ui.render_blocks</code>
FR-14 UI and sources	<code>app/ui.py</code>
FR-15 incremental ingestion	<code>writer.py</code> , <code>cli.py ingest add/replace/remove</code>
FR-16 evaluation	<code>eval/</code> (RAGAS + custom metrics), <code>eval_runs</code> , <code>eval_results</code> , <code>query_log</code>
NFR-9 consistency	<code>writer.write_document</code> (one transaction)
NFR-10 recoverability	caches (§7), backups (plan Phase 8)
FR-17 sign-in	<code>auth.service.login</code> , <code>require_user</code> , <code>app/login.py</code>
FR-18 account admin	<code>cli.py user ...</code>
FR-19 per-user limits	<code>auth.service.check_limits</code> , <code>query_log</code>
FR-20 history and audit	<code>query_log.user_id / answer_json</code> , <code>auth_events</code>
NFR-11 credential security	<code>auth/passwords.py</code> , <code>auth/sessions.py</code> , §9
FR-21 end-to-end tracing	<code>obs/telemetry.py</code> , span tree (§5.9)
FR-22 admin view	admin page in <code>app/</code> , <code>query_log</code> , metrics
FR-23 multi-turn threads	LangGraph Postgres checkpointer, <code>thread_id</code> (§5.2)
FR-24 long-term memory	<code>memory/</code> (§5.11), <code>recall_memory / remember</code> nodes
FR-25 memory controls	<code>memory.list/delete/set_enabled</code> , CLI then UI
NFR-13 memory privacy	per-user namespaces, telemetry content switch, retention (§5.11)
NFR-12 telemetry safety	batch exporter, content capture switch, retention (§5.9)

12. Changelog

Version	Date	Changes
0.1	2026-09-26	Initial LLD
0.2	2026-09-26	Review fixes: citation IDs defined (§3.1); hybrid search aggregates element locations, one row per chunk (§5.4); local Docker setup with a named volume (§4.2; indexes renumbered to §4.3); Streamlit session-state rules (§5.7); new integration tests (§10).
0.3	2026-09-26	Email + password sign-in: <code>auth</code> modules and <code>app/login.py</code> (§1); <code>auth.*</code> config (§2); <code>user</code> CLI commands (§3.9); <code>users</code> , <code>sessions</code> , <code>auth_events</code> tables, <code>query_log.user_id / answer_json</code> , and indexes (§4); sign-in step in the request lifecycle (§5.1); new §5.8 authentication; error, logging, security, test and traceability updates.

Version	Date	Changes
0.4	2026-09-26	OpenTelemetry observability: <code>obs/</code> modules (§1); <code>observability.*</code> and <code>retention.*</code> config (§2); <code>obs cleanup</code> command (§3.9); <code>query_log.trace_id</code> (§4); new §5.9 (setup, span tree, other traces, metrics, content capture, correlation, export, retention, failure safety); error, logging, security, test and traceability updates.
0.5	2026-09-26	RAGAS evaluation: <code>eval/</code> modules (§1); <code>eval.*</code> config (§2); <code>eval</code> CLI commands (§3.9); <code>eval_runs</code> and <code>eval_results</code> tables (§4); new §5.10 (golden set, run steps, RAGAS samples and metrics, custom metrics, regression gate, report, judge reliability); regression test and traceability updates.
0.6	2026-09-27	§10 testing strategy and tooling (TDD where it fits, frozen regression tests, mocked OpenAI, <code>mmrag_test</code> database, pytest markers, CI), following constitution W5.
0.7	2026-09-29	Agent on LangGraph (D3): <code>agent/graph.py</code> replaces <code>agent/loop.py</code> (§1, §5.1, §5.2 state, nodes and edges, checkpoint). Long-term memory module and §5.11 (Phase 9). §3.5 and §4: <code>element_links</code> for figures and tables, similarity-checked rule 3. §5.4: related items. Tools stay in-process; no MCP (D16). Traceability for FR-23–25 and NFR-13.